

The background features a light beige grid pattern. Various food items are illustrated in a hand-drawn style: a bowl of ramen with chopsticks in the top left, two fried eggs in the top right, a slice of orange with a star-shaped garnish on the left, two bread rolls with tomato and onion in the bottom left, and a slice of pizza in the bottom center.

PIZZA SALES ANALYSIS

SUPRIYA C M
DATA ANALYST

INTRODUCTION



The food industry, particularly fast food chains, relies heavily on data-driven decisions to enhance customer satisfaction and maximize profitability. In this context, sales analysis plays a crucial role in understanding product demand, revenue distribution, and business performance. This project focuses on analyzing pizza sales data using SQL, providing valuable insights into order patterns, product popularity, and revenue contribution across different pizza categories.

By leveraging SQL's powerful querying capabilities, we explore the relationships between orders, pizza types, and sales figures. This analysis enables businesses to identify best-selling products, optimize inventory, and design effective marketing strategies. The project aims to transform raw sales data into actionable insights, driving informed decision-making for business growth.



OBJECTIVE



Designed and executed a Pizza Sales Analysis project using SQL to uncover key business insights and support data-driven decisions. Analyzed sales data by performing complex queries involving multiple joins, aggregations, subqueries, and window functions. Calculated category-wise revenue contribution, total sales, and order frequency while identifying top-selling pizza types and seasonal demand patterns. Ensured data accuracy and optimized query performance for efficient analysis. Delivered a detailed report on revenue trends and customer preferences, providing valuable recommendations for inventory and marketing strategies.



Software -MySQL Workbench

DATASET OVERVIEW

ORDERS

	order_id	order_date	order_time
▶	1	2015-01-01	11:38:36
	2	2015-01-01	11:57:40
	3	2015-01-01	12:12:28
	4	2015-01-01	12:16:31
	5	2015-01-01	12:21:30
	6	2015-01-01	12:29:36

PIZZAS

	pizza_id	pizza_type_id	size	price
▶	bbq_ckn_s	bbq_ckn	S	12.75
	bbq_ckn_m	bbq_ckn	M	16.75
	bbq_ckn_l	bbq_ckn	L	20.75
	cali_ckn_s	cali_ckn	S	12.75
	cali_ckn_m	cali_ckn	M	16.75
	cali_ckn_l	cali_ckn	L	20.75

ORDERS_DETAILS

	order_details_id	order_id	pizza_id	quantity
▶	1	1	hawaiian_m	1
	2	2	classic_dlx_m	1
	3	2	five_cheese_l	1
	4	2	ital_supr_l	1
	5	2	mexicana_m	1
	6	2	thai_ckn_l	1

PIZZA_TYPES

	pizza_type_id	name	category	ingredients
▶	bbq_ckn	The Barbecue Chicken Pizza	Chicken	Barbecued Chicken, Red Peppers, Green Peppe...
	cali_ckn	The California Chicken Pizza	Chicken	Chicken, Artichoke, Spinach, Garlic, Jalapeno P...
	ckn_alfredo	The Chicken Alfredo Pizza	Chicken	Chicken, Red Onions, Red Peppers, Mushrooms...
	ckn_pesto	The Chicken Pesto Pizza	Chicken	Chicken, Tomatoes, Red Peppers, Spinach, Garl...
	southw_ckn	The Southwest Chicken Pizza	Chicken	Chicken, Tomatoes, Red Peppers, Red Onions, ...
	thai_ckn	The Thai Chicken Pizza	Chicken	Chicken, Pineapple, Tomatoes, Red Peppers, T...
	big_meat	The Big Meat Pizza	Classic	Bacon, Pepperoni, Italian Sausage, Chorizo Sau...

RETRIEVE THE TOTAL NUMBET OF ORDER PLACED

```
SELECT  
    COUNT(order_id) AS Total_Orders  
FROM  
    orders;
```

	Total_Orders
▶	21350

CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES

```
SELECT
    ROUND(SUM(orders_details.quantity * pizzas.price),
          2) AS Total_Sales
FROM
    orders_details
    JOIN
    pizzas ON orders_details.pizza_id = pizzas.pizza_id;
```

	Total_Sales
▶	817860.05

IDENTIFY THE HIGHEST PRICED PIZZA

```
SELECT
    pizza_types.name, pizzas.price
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```

Result Grid			 Filter Rows:	
	name	price		
▶	The Greek Pizza	35.95		

IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED

```
SELECT
    pizzas.size, COUNT(orders_details.quantity)
FROM
    pizzas
    JOIN
    orders_details ON pizzas.pizza_id = orders_details.pizza_id
GROUP BY size
ORDER BY size;
```

Result Grid				 Filter Rows:
	size	count(orders_details.quantity)		
▶	L	18526		
	M	15385		
	S	14137		
	XL	544		
	XXL	28		

IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED

```
SELECT
    pizza_types.name, SUM(orders_details.quantity) AS quantity
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
    JOIN
    orders_details ON orders_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY quantity DESC
LIMIT 5;
```

Result Grid			Filter Rows:
	name	quantity	
▶	The Classic Deluxe Pizza	2453	
	The Barbecue Chicken Pizza	2432	
	The Hawaiian Pizza	2422	
	The Pepperoni Pizza	2418	
	The Thai Chicken Pizza	2371	

JOIN NECESSARY TABLE TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED

```
select pizza_types.category,  
sum(orders_details.quantity) as Total_quantity  
from pizza_types join pizzas  
on pizza_types.pizza_type_id = pizzas.pizza_type_id  
join orders_details on orders_details.pizza_id = pizzas.pizza_id  
group by category  
order by Total_quantity desc;
```

Result Grid |   Filter Rows:

	category	Total_quantity
▶	Classic	14888
	Supreme	11987
	Veggie	11649
	Chicken	11050

DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR IN A DAY

```
SELECT
```

```
    HOUR(order_time) AS hour, COUNT(order_id) AS order_count
```

```
FROM
```

```
    orders
```

```
GROUP BY HOUR(order_time);
```

Result Grid |   Fil

hour	order_count
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28
10	8

JOIN THE RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS

```
SELECT  
    category, COUNT(name)  
FROM  
    pizza_types  
GROUP BY category;
```

Result Grid			Filter Rows:
	category	count(name)	
▶	Chicken	6	
	Classic	8	
	Supreme	9	
	Veggie	9	

GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY

```
SELECT
    AVG(quantity)
FROM
    (SELECT
        orders.order_date, SUM(orders_details.quantity) AS quantity
    FROM
        orders
    JOIN orders_details ON orders.order_id = orders_details.order_id
    GROUP BY order_date) AS AVG_order;
```

Result Grid		Filter Rows:	
	avg(quantity)		
▶	138.4749		

GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY

```
SELECT
    pizza_types.name,
    ROUND(SUM(orders_details.quantity * pizzas.price),
          0) AS revenue
FROM
    pizza_types
    JOIN
    pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
    JOIN
    orders_details ON pizzas.pizza_id = orders_details.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue DESC
LIMIT 3;
```

Result Grid   Filter Rows:		
	name	revenue
▶	The Thai Chicken Pizza	43434
	The Barbecue Chicken Pizza	42768
	The California Chicken Pizza	41410

CALCULATE THE PERCENTAGE CONTRIBUTION OF EACH PIZZA TYPE TO TOTAL REVENUE

```
SELECT pizza_types.category,  
       (SUM(orders_details.quantity * pizzas.price) /  
        (SELECT ROUND(SUM(orders_details.quantity * pizzas.price), 2)  
         FROM pizza_types  
         JOIN pizzas  
           ON pizza_types.pizza_type_id = pizzas.pizza_type_id  
         JOIN orders_details  
           ON pizzas.pizza_id = orders_details.pizza_id)) * 100 AS revenue_percentage  
FROM pizza_types  
JOIN pizzas  
  ON pizza_types.pizza_type_id = pizzas.pizza_type_id  
JOIN orders_details  
  ON pizzas.pizza_id = orders_details.pizza_id  
GROUP BY pizza_types.category  
ORDER BY revenue_percentage DESC;
```

Result Grid			 Filter Rows:
	category	revenue_percentage	
▶	Classic	26.90596025566967	
	Supreme	25.45631126009862	
	Chicken	23.955137556847287	
	Veggie	23.682590927384577	

ANALYZE THE CUMULATIVE REVENUE GENERATED OVER TIME

```
select order_date,  
sum(revenue) over (order by order_date) as CUM_revenue  
from  
(select orders.order_date,  
sum(orders_details.quantity*pizzas.price) as revenue  
from orders_details join pizzas  
on pizzas.pizza_id = orders_details.pizza_id  
join orders  
on orders.order_id = orders_details.order_id  
group by orders.order_date) as Sales;
```

Result Grid  Filter Rows:		
	order_date	CUM_revenue
▶	2015-01-01	2713.8500000000004
	2015-01-02	5445.75
	2015-01-03	8108.15
	2015-01-04	9863.6
	2015-01-05	11929.55
	2015-01-06	14358.5
	2015-01-07	16560.7
	2015-01-08	19399.05
	2015-01-09	21526.4
	2015-01-10	23990.350000000002
	2015-01-11	25862.65
	2015-01-12	27781.7
	2015-01-13	29831.300000000003

DETERMINE THE TOP 3 MOST ORDERD PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY

```
select name, revenue from
(select category, revenue, name,
rank() over(partition by category order by revenue desc ) as rn
from
(select pizza_types.category ,pizza_types.name,
sum((orders_details.quantity)*pizzas.price) as revenue
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join orders_details
on orders_details.pizza_id = pizzas.pizza_id
group by pizza_types.category, pizza_types.name) as a) as b
where rn <= 3;
```

Result Grid			Filter Rows:	E
	name	revenue		
▶	The Thai Chicken Pizza	43434.25		
	The Barbecue Chicken Pizza	42768		
	The California Chicken Pizza	41409.5		
	The Classic Deluxe Pizza	38180.5		
	The Hawaiian Pizza	32273.25		
	The Pepperoni Pizza	30161.75		
	The Spicy Italian Pizza	34831.25		
	The Italian Supreme Pizza	33476.75		
	The Sicilian Pizza	30940.5		
	The Four Cheese Pizza	32265.700000000065		
	The Mexicana Pizza	26780.75		
	The Five Cheese Pizza	26066.5		

CONCLUSION

The Pizza Sales Analysis project provided valuable insights into sales performance and product demand through efficient SQL-based data exploration and analysis. By analyzing order details, revenue distribution, and category-wise sales, we identified top-performing pizza types and their contribution to overall business revenue. This analysis helps in understanding customer preferences, enabling better inventory management and more effective marketing strategies.

Furthermore, the project demonstrated the importance of SQL in extracting and transforming data to generate meaningful business insights. The approach used in this project can be extended to identify seasonal sales trends, optimize pricing strategies, and forecast future demand. With advanced visualizations and deeper customer segmentation, this analysis can evolve into a comprehensive business intelligence solution.